

Proyecto Final: Técnicas Actuales

Introducción

En éste proyecto final se les propone poner manos a la obra en utilizar alguna herramienta de machine learning del estado del arte¹. Para ésto se presentan algunos modelos de redes de uso actual que funcionan en diversos dominios (datos numéricos, lenguaje natural, imágenes, audio, etc.)

Presentaremos modelos que permiten el uso de técnicas como in-context-learning o fine-tuning, que nos permitirán evitar tener que entrenar los modelos desde cero (que puede ser muy costoso tanto en datos como en recursos computacionales).

En el caso de los modelos con in-context-learning se espera que el alumno pueda proveer datos como contexto de forma de que el modelo de una respuesta esperada, e interprete las respuestas en el contexto dado, explorando cómo distintos contextos pueden devenir en distintas respuestas.

Por otro lado en el caso de las redes con fine-tuning se espera que el alumno pueda pre-procesar los datos y entrenar las capas necesarias en la red de forma que las métricas usuales cumplan con requisitos esperados para luego poder correr la red (clasificar o regresar) sobre datos nuevos efectivamente.

Consigna

Se espera que se ponga en práctica uno de los métodos de las publicaciones que se presentan a continuación, y se entregue un informe del trabajo realizado en el cual se exhiban los siguientes puntos:

- Análisis del conjunto de datos: tipo de información, preprocesado de los datos (si se ha hecho o si viene hecho), campos presentes (si los hay), distribución de los mismos.
- Interpretación de la publicación: resumen del método propuesto, consideraciones más relevantes realizadas por los autores.
- Análisis conjunto de datos y método: posibilidades de aplicación del método a los datos, expectativas de resultados.
- Metodología de la implementación: uso de la implementación, cambios realizados (si hubo), método de entrenamiento, elección de parámetros.
- Resultados obtenidos: descripción de los experimentos, análisis cuantitativo y cualitativo de resultados, resultados con distintos parámetros.
- Discusión de resultados, limitaciones, y posibilidades de trabajo futuro.

Además se pide la entrega del código usado, ya sea en Notebook o en un repositorio autocontenido, funcionando.

En las próximas páginas encontrará las propuestas posibles, seleccione una de ellas.

¹ “«tecnología punta», «lo último en tecnología» o «tecnología de vanguardia».” - ([Wikipedia](#))

Proyecto Final: Técnicas Actuales

Opción 1: In-Context Learning con LLaMA

In-Context learning es una forma de aprender específica para grandes modelos de lenguaje (LLM) que conlleva darle al modelo una serie de elementos a tener en cuenta (context) y una tarea a resolver (prompt) y se espera que pueda resolverlo utilizando toda la información con la que fue entrenado. El problema a resolver no está entre lo que el modelo aprendió, sino que se espera que pueda extraer el conocimiento que se da en el prompt, junto a lo que ya conoce, para realizar la tarea que se pide.

En este proyecto utilizaremos LLaMA (Large Language Model Meta AI), un gran modelo de lenguaje decoder-only, pre-entrenado por Meta.

 "Language Models are Few-Shot Learners", Brown et al. (2020)

 <https://arxiv.org/abs/2005.14165>

 Modelo en HuggingFace:

 [meta-llama/Llama-2-7b](https://huggingface.co/meta-llama/Llama-2-7b) , https://huggingface.co/docs/transformers/en/model_doc/llama

Exploraciones

Se espera que el alumno realice experimentos sobre el modelo provisto y la tarea a resolver elegida, revisando sus resultados ante distintos contextos provistos. Se muestra un ejemplo a continuación para que sirva de guía:

En nuestro experimento, intentaremos hacer que LLaMA prediga el sentimiento (positivo/negativo) de distintos conjuntos de texto del dataset de ratings de películas IMDB. En particular buscamos darle elementos (reviews) con distinto grado de positividad o negatividad, junto con su respuesta correcta, y luego analizar cómo actúa ante elementos de un estilo similar, o de un estilo completamente distinto. Como nuestro conjunto de datos tiene alguna valoración del sentimiento (en este caso ratings de películas, el valor en estrellas dado), podemos buscar generar alguna métrica que automáticamente nos diga qué tan bueno es nuestro predictor.

Para pensar: ¿Qué pasa si le doy un contexto totalmente equivocado?

En segundo lugar probaremos darle distintas cantidad de contexto, los casos en que no le damos contexto se llaman zero-shot, los casos en que les doy un sólo ejemplo se llaman one-shot, los casos en que les doy muy pocos casos en contexto se llaman few-shot, y el resto many-shot.

Para pensar: ¿Cómo mejora el modelo original sin contexto del contextualizado? ¿Qué pasa si el contexto único es positivo o si es negativo?

Además la publicación original tiene ejemplos de dónde acierta y cuando falla el modelo, compararemos con lo obtenido.

Proyecto Final: Técnicas Actuales

Opción 2: Zero-Shot Learning con CLIP

CLIP (Contrastive Language–Image Pretraining) es una red neuronal entrenada por OpenAI para conectar imágenes y texto. Dada una imagen o un prompt, CLIP lleva cualquiera de ellos a un espacio métrico común, en el cual se puede medir la distancia entre ambos. Esto hace que CLIP sea un zero-shot learner, es decir, no requiere entrenamiento para adaptarse a nuevas tareas.

Buscaremos enmarcar una tarea de clasificación de imágenes o búsqueda de imágenes a partir de texto utilizando CLIP.

 "Learning Transferable Visual Models From Natural Language Supervision", Radford et al. (2021)
 <https://arxiv.org/abs/2103.00020>

 Modelo en HuggingFace:
 <https://huggingface.co/openai/clip-vit-base-patch32>

Exploraciones

Se espera que el alumno pueda enmarcar el problema planteado como un problema conjunto de imágenes y texto, utilizando CLIP para codificar ambos a un espacio común, y revisando sus resultados ante distintos tipos de contextos provistos. Se muestra un ejemplo a continuación para que sirva de guía:

En primer lugar intentaremos hacer que CLIP clasifique un conjunto de imágenes en categorías dadas por nosotros. En particular se busca ver algunos casos interesantes como son: categorías con las que no fue entrenado, categorías con las que sí fue entrenado y un conjunto mezcla de ambos casos. Puede ser interesante probar los 3 casos sobre el mismo conjunto de datos o sobre distintos conjuntos de datos, plantee su elección y sus hipótesis.
Para pensar: ¿Cómo afecta la forma en que describen las categorías a la clasificación final? ¿Qué sucede si todas (o todas menos una) las categorías dadas no tienen relación con las imágenes dadas?

En segundo lugar buscaremos que CLIP encuentre la (o las) imágenes que más se acerque a nuestra descripción textual. En este caso estaremos explorando cómo CLIP asocia imágenes y texto en un espacio común a ambos.
Para pensar: ¿qué tan específico o que tan vaga puedo hacer a mi descripción sin que CLIP falle en encontrar la imagen correcta?

Además OpenAI provee descripciones ("prompts") específicas con las que saben que CLIP funciona correctamente. Probaremos utilizarlas y comparar los resultados con las propias.

Proyecto Final: Técnicas Actuales

Opción 3: Fine-Tuning de Modelos con BERT

BERT (Bidirectional Encoder Representations from Transformers) es un modelo de lenguaje basado en transformers. Su novedad es condicionar la generación de palabras tanto a derecha como a izquierda (la B de BERT es por “bidireccional”) y tener una arquitectura particularmente útil para realizar fine-tuning. Utilizaremos un modelo asociado, llamado DistilBERT (Distilled Version of BERT), reducido en tamaño y con mayor velocidad, pero de similares características, y le agregaremos una capa final que entrenaremos para adaptar el conocimiento que posee a nuestro problema de clasificación particular.

 "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", J. Devlin et al. (2018)

 <https://arxiv.org/abs/1810.04805>

 "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter", V. Sahn et al. (2019)

 <https://arxiv.org/abs/1910.01108>

 Modelos en HuggingFace:

 [bert-base-uncased](#) , [distilbert-base-uncased](#)

Exploraciones

Se espera que el alumno realice las modificaciones necesarias para adaptar el modelo a la tarea a resolver elegida, lo ponga en práctica y analice los resultados obtenidos. Se muestra un ejemplo a continuación para que sirva de guía:

Como uso de BERT planteamos la idea de predecir el sentimiento (positivo/negativo) de distintos conjuntos de texto del dataset de ratings de películas IMDB. En particular nos interesa ver cómo funciona antes de realizarle un fine-tuning, y cómo funciona luego.

Para pensar: ¿Se equivoca en algún ejemplo en el que pre-tuning no se equivocaba?

Luego nos planteamos analizar si el fine-tuning que realizamos ha causado uno de los problemas conocidos en ML como “catastrophic forgetting”, es decir, que al realizar fine-tuning para un problema, hemos causado que la red deje de funcionar o funcione peor para otros problemas. En este caso tomaremos el dataset AG News, y analizaremos cómo funciona la red para clasificación en el mismo antes y después de nuestro fine-tuning con IMDB.

Para pensar: ¿Qué pasa si variamos la cantidad de épocas o la cantidad de datos que le pasamos al fine-tuning?

Adicionalmente podríamos probar cómo funciona el modelo BERT de tamaño completo contra el modelo DistilBERT entrenado por nosotros.

Proyecto Final: Técnicas Actuales

Opción 4: Fine-Tuning con InceptionV3

Inception es una familia de redes profunda convolucionales desarrolladas por Google para la clasificación de imágenes. Las mismas están pre-entrenadas en ImageNet, siendo InceptionV3 la más moderna y más utilizada para realizar transfer-learning, utilizando sus capas convolucionales base como un poderoso extractor de características, adaptable a nuevas tareas. Aplicaremos fine-tuning a las últimas capas de la red para adaptarla a nuestro problema a resolver.

 "Rethinking the Inception Architecture for Computer Vision", C. Szegedy et al. (2015)
 <http://arxiv.org/abs/1512.00567>

 Modelo en HuggingFace:
 [inception_v3](#)

 Modelo en Keras:
 <https://keras.io/api/applications/inceptionv3/> , https://keras.io/guides/transfer_learning/

 Modelo en PyTorch:
 <https://docs.pytorch.org/vision/main/models/inception.html>

Exploraciones

Se espera que el alumno realice un entrenamiento sobre las últimas capas del modelo provisto para adaptarla a la tarea a resolver elegida, explorando distintos parámetros, comparando distintos métodos y analizando los resultados obtenidos. Se muestra un ejemplo a continuación para que sirva de guía:

Nos interesa clasificar las imágenes del dataset CIFAR-10, que consiste en 60000 imágenes pertenecientes a 10 clases distintas. Para esto mantendremos toda la red como se encuentra, “congelando” todas las capas menos las últimas, quitando la última que predice 1001 clases y reemplazándola por una capa propia, que predice 10 clases distintas.

Para pensar: ¿Cómo cambia el entrenamiento si inicializo esa última capa de distintas formas? ¿Hay clases similares en el output original?

Por otro lado nos interesa explorar hasta qué profundidad se puede descongelar la red y entrenar para un dataset pequeño como es CIFAR-10 sin que ocurra el overfitting, o si agregar épocas aún con menos capas entrenables también produce overfitting.

Para pensar: ¿Cuántos pesos estoy entrenando con cada capa adicional que descongelo?

Adicionalmente, puede ser interesante ver qué outputs están relacionados, por ejemplo si le paso todas las imágenes de una clase, ¿cuáles outputs son los que más se activan junto al correcto?.